

# Attention Score Acceleration using an FPGA

Samarth, Rijul, Om and Satyarth

RC19

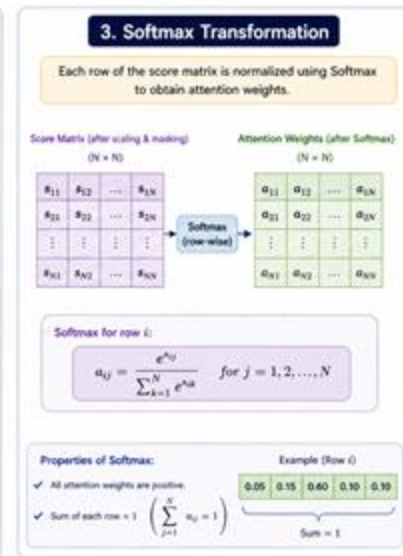
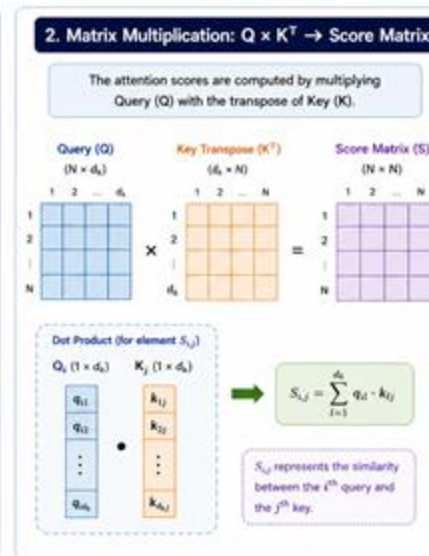
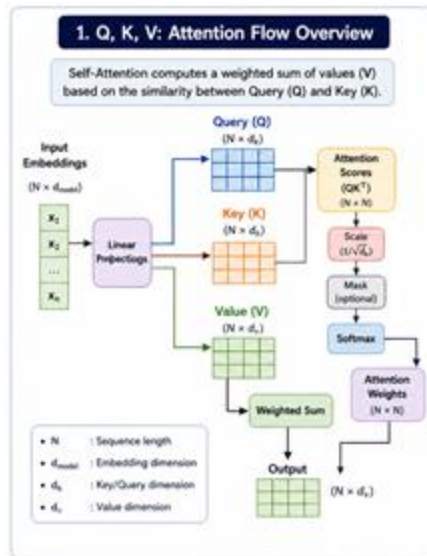
# Project Overview

- Built a standalone one-head attention accelerator for the AMD/Xilinx U55C FPGA.
- The FPGA takes one attention head's query, key, and value data from TinyLlama and produces the corresponding one-head attention output.
- Verified synthetic sequence lengths  $S = 8, 64, 128, 256, 512$  and real TinyLlama vectors.



# Background

- Transformer models rely on the **self-attention mechanism** to capture relationships between tokens in a sequence
- Input embeddings are projected into **Query (Q), Key (K), and Value (V)** representations
- Attention scores are computed using a **matrix multiplication:  $Q \times K^T$** , which measures similarity between tokens
- The scores are **scaled, optionally masked, and passed through a softmax function** to produce normalized attention weights
- These weights are used to compute a **weighted combination of Value (V) vectors**, forming the final attention output
- The  **$Q \times K^T$  operation dominates computation**, with complexity  $O(N^2d)$ , making it the primary performance bottleneck



# Motivation

Edge LLMs need  
efficient local inference.

Attention cost grows  
quickly with sequence  
length.

FPGAs can exploit  
custom dataflow,  
parallelism, quantization,  
and HBM.

We isolate one TinyLlama  
attention head to measure  
correctness, resource use,  
and performance on real  
U55C hardware.

# Methodology & Implementation

## Fused Score + Mask/Scale

Computes  $QK^T$ , applies causal mask, and scales logits in one kernel.

## Full-Row Softmax

Normalizes across all S keys, required for  $S > 64$ .

## softmax @ V

Consumes attention weights and V tiles to produce one-head attn\_out.

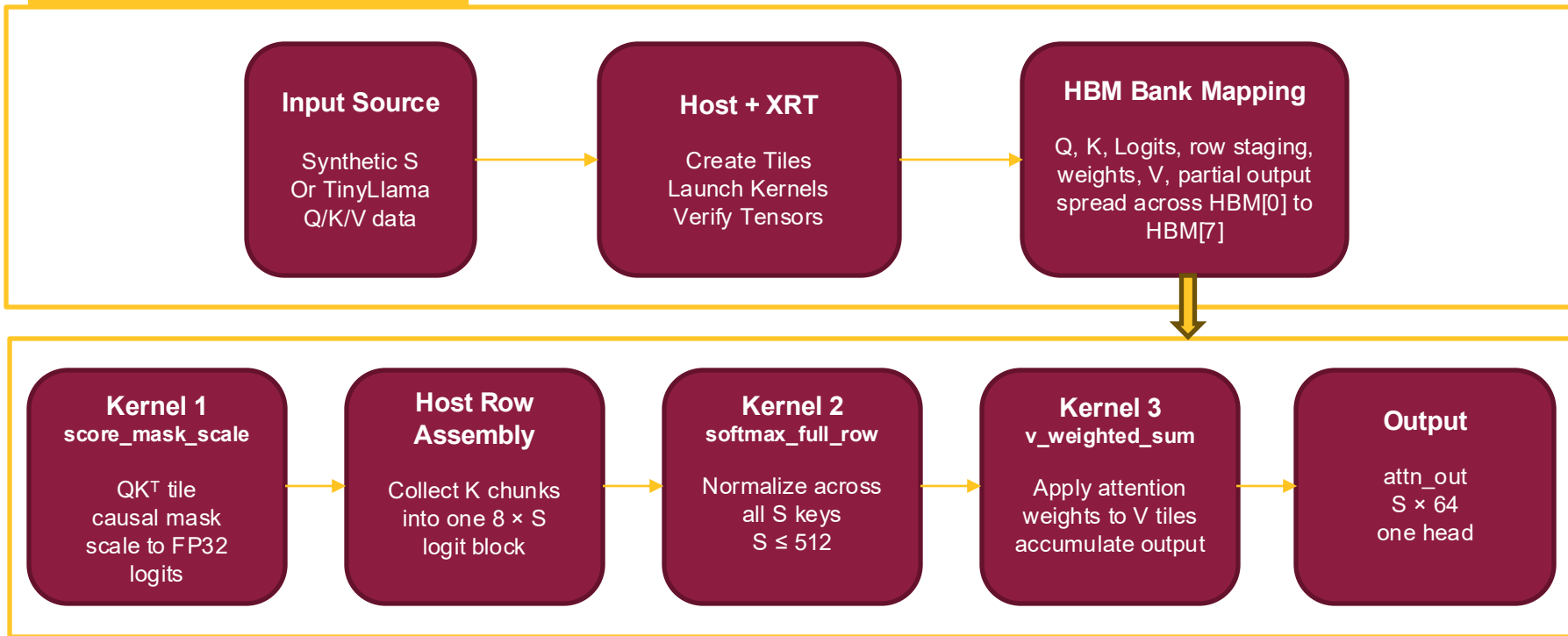
## Tiling

Q chunks: 8 rows. K/V chunks: 64 columns. Supports S up to 512.

## Verification:

Host compares logits, softmax weights, and attn\_out against Python references.

# Architecture



## HBM layout



# Verification Strategy

Python reference verifies full-sequence tiling at S=8, 64, 128, 256, 512

Each HLS kernel tested independently against exported vectors. All pass locally before touching hardware.

Hardware emulation, to test interaction between host and device:

- Fused hardware emulation passed vector mode and S=8 sequence mode

On-board execution:

- Synthetic sweep passed S=8, 64, 128, 256, 512
- TinyLlama vector runs passed for real S = 8, 64, 128

```
advent@advent:~/Desktop/RC19/attention_score_u55c$ ./build/host_attention_score_chain --xclbin build/attention_score_chain.xclbin --seq-len 512 --device 0
Opening device 0
Running tiled synthetic sequence, S=512, q_chunks=64, k_chunks=8
Kernel timing summary (host wall-clock, launch through wait):
  score_mask_scale_u55c_kernel15.448 ms
  softmax_full_row_u55c_kernel8.266 ms
  v_weighted_sum_u55c_kernel 17.485 ms
  kernel_launch_wait_sum 41.200 ms
  host_dma_sync_gap 13.061 ms
  total_chain 54.261 ms
Tiled sequence verification PASSED
Attention output verification PASSED
XRT chain verification PASSED
```

# Current status

| Track                          | Status                                                    | Notes                                                                                                                              |
|--------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Track A: speedup + tiling      | Steps 1–5 hardware-verified                               | Includes packed I/O, merged mask+scale, tiling, full-row softmax, pass-1 double buffering, and fused score+mask/scale.             |
| Track B: softmax @ V           | Complete                                                  | V weighted-sum kernel integrated; synthetic S=8..512 and vector modes verify final attn_out.                                       |
| Track C: real TinyLlama inputs | Complete for current scope                                | TinyLlama Q/K/V extraction, INT8 Q/K quantization, real-vector export, and U55C runs for single-tile, S=16, S=64.                  |
| Track D: baselines/results     | Staged design complete; fused retiming partially captured | Fused hardware timings are available from AGENTS/U55C run logs; Track D tables should be refreshed for fused-vs-staged comparison. |
| Future integration             | Not implemented                                           | Full TinyLlama subgraph, KV-cache-aware decode, and decoder-layer integration remain future work.                                  |



# Results

|                                       |                                      |
|---------------------------------------|--------------------------------------|
| Synthetic S = 8, 64, 128, 256, 512    | ✓ PASSED                             |
| Real TinyLlama single tile (8 tokens) | ✓ PASSED                             |
| Real TinyLlama S = 16                 | ✓ PASSED                             |
| Real TinyLlama S = 64                 | ✓ PASSED                             |
| attn_out matches PyTorch within 1e-3  | ✓ PASSED                             |
| Post-route timing closure             | ✓ WNS 0.003 ns · 0 failing endpoints |

Every run printed: Tiled sequence verification PASSED · Attention output verification PASSED · XRT chain verification PASSED

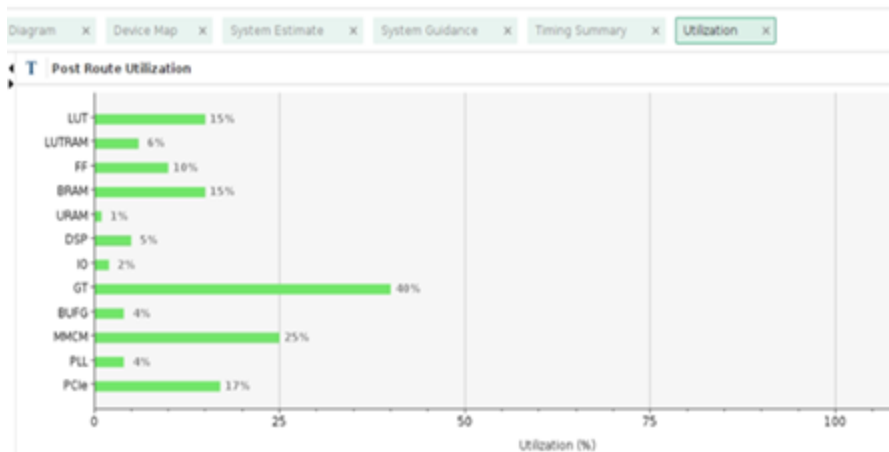
# CPU/GPU/FPGA Comparison

| S   | CPU full attn (ms) | GPU full attn (ms) | Staged FPGA total (ms) | Fused FPGA total (ms) |
|-----|--------------------|--------------------|------------------------|-----------------------|
| 8   | 0.0322             | 0.3349             | 0.803                  | 0.296                 |
| 64  | 0.2920             | 0.2507             | 2.864                  | 2.364                 |
| 128 | 2.5009             | 0.2998             | 6.992                  | 5.484                 |
| 256 | 3.9523             | 0.2726             | 20.570                 | 17.307                |
| 512 | 12.3290            | 0.2461             | 72.941                 | 60.328                |

- The FPGA path is correct, but still slower than CPU/GPU because the design uses many tile-level launches and host-visible HBM transfers.

# Utilization Report

- Full routed design: 14.70% LUTs, 9.72% FF, 15.35% BRAM, 0.21% URAM, 4.53% DSP.
- The V weighted-sum dominates DSP usage: 320 DSP.
- Post-route timing closed: WNS 0.003 ns at kernel target clock: 300 Mhz.



| Resource | Utilization | Available | Utilization % |
|----------|-------------|-----------|---------------|
| LUT      | 191648      | 1303680   | 14.70         |
| LUTRAM   | 36960       | 600960    | 6.15          |
| FF       | 253384      | 2607360   | 9.72          |
| BRAM     | 309.50      | 2016      | 15.35         |
| URAM     | 2           | 960       | 0.21          |
| DSP      | 409         | 9024      | 4.53          |
| IO       | 12          | 624       | 1.92          |
| GT       | 16          | 40        | 40.00         |
| BUFG     | 42          | 1008      | 4.17          |
| MMCM     | 3           | 12        | 25.00         |
| PLL      | 1           | 24        | 4.17          |
| PCle     | 1           | 6         | 16.67         |

Table: Post-route utilization report

# Challenges and Limitations

## Challenges

### Debugging HLS Kernels

- Limited visibility compared to software debugging
- Requires waveform or simulation-based debugging

### Long Compilation Times

- FPGA synthesis takes a long time, slowing iteration

### Precision Trade-Offs

- Balancing accuracy with performance when using INT8/FP32

## Limitations

- Standalone one-head block only

- Not yet integrated to full TinyLlama decoder.

- No KV-cache decode flow yet

- Slower than CPU and GPU baseline

# Next Steps

## Reduce Host Transfers:

- Keep Q/K/V, logits, probabilities, and attn\_out resident on FPGA memory.

## Reduce Kernel Launch Count

- Move from many tile-level launches toward larger fused kernels.

## Fuse Softmax and V accumulation

- Avoid writing full attention weights back through host-visible buffers.

## Full Model Integration

- Connect the accelerated block into TinyLlama decoder flow after performance improves.

# References

- <http://neuralnetworksanddeeplearning.com/>
- <https://jalammar.github.io/illustrated-transformer/>
- <https://medium.com/@cristianleo120/the-math-behind-transformers-6d7710682a1f>



# THANK YOU!

Any Questions?